

Project - Low Level Design

on

Entertainment Planning Assistant Agent

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	PRABAL SHUKLA	EN22CS301713
2.	PRATIK KOUSHAL	EN22CS301747
3.	PRIYANSH CHOUHAN	EN22CS301759
4.	PUSHPRAJ SINGH CHOUHAN	EN22CS301774
5.	RAJAT MALVIYA	EN22CS301783

*Group Name:*11 D7

*Project Number:*AAI-34

Industry Mentor Name: Aashruti Shah

University Mentor Name: Ajeet Singh Rajput

*Academic Year:*2025-2026

Table of Contents

1. **Introduction.**
 - 1.1. Scope of the document.
 - 1.2. Intended Audience
 - 1.3. System overview.
2. **System Design.**
 - 2.1. Application Design
 - 2.2. Process Flow.
 - 2.3. Information Flow.
 - 2.4. Components Design
 - 2.5. Key Design Considerations
 - 2.6. API Catalogue.
3. **References**

1. Introduction

The Entertainment Planning Assistant Agent is an intelligent system designed to assist users in planning entertainment activities such as watching movies, attending events, or exploring sports activities in a specific city. The system uses an agentic AI architecture where a planner agent receives user inputs and generates recommendations using a Large Language Model (LLM).

The primary purpose of this project is to demonstrate how an agent-based architecture can be used to process user requests and generate context-aware responses. The system integrates a backend API built with FastAPI, a user interface developed using Streamlit, and a language model accessed through the Groq API.

The application receives a user request containing a movie or entertainment option along with a city. The backend agent processes this request, generates a prompt, and sends it to the LLM. The LLM analyzes the input and produces a recommendation that includes possible theatres, show timings, approximate ticket prices, and public opinion.

The system follows a modular architecture where each module has a clear responsibility, allowing easy maintenance and scalability.

1.1 Scope of the Document

This document describes the Low Level Design (LLD) of the Entertainment Planning Assistant Agent system. It explains the internal architecture, module structure, and detailed design of the application components.

The scope of this document includes the description of the backend architecture, application modules, API endpoints, and the interaction between different components of the system. It also outlines the data flow and processing logic used to generate entertainment recommendations.

The document focuses specifically on the internal design and implementation details of the system rather than high-level architectural concepts.

1.2 Intended Audience

This document is intended for developers, project reviewers, and academic evaluators who require a detailed understanding of how the system is implemented internally.

The intended audience includes software developers responsible for implementing or maintaining the system, faculty members evaluating the project design, and technical reviewers interested in understanding the functioning of the agent-based architecture.

1.3 System Overview

The Entertainment Planning Assistant Agent is a backend-driven application that allows users to generate entertainment plans based on their preferences and location.

The system consists of three main layers:

Frontend Layer

The frontend interface is implemented using Streamlit and provides a simple graphical interface where users can enter the movie or event name and the city.

Backend Layer

The backend is developed using FastAPI. It receives requests from the frontend, invokes the planner agent, and returns the generated entertainment plan.

AI Agent Layer

The Planner Agent is responsible for interpreting the user request and generating a prompt for the language model. The Groq LLM processes this prompt and generates recommendations.

The system workflow begins when the user submits a request through the frontend interface. The request is sent to the backend API, which invokes the planner agent. The agent generates a prompt that is sent to the LLM through the Groq API. The response generated by the LLM is returned to the backend and displayed on the frontend interface.

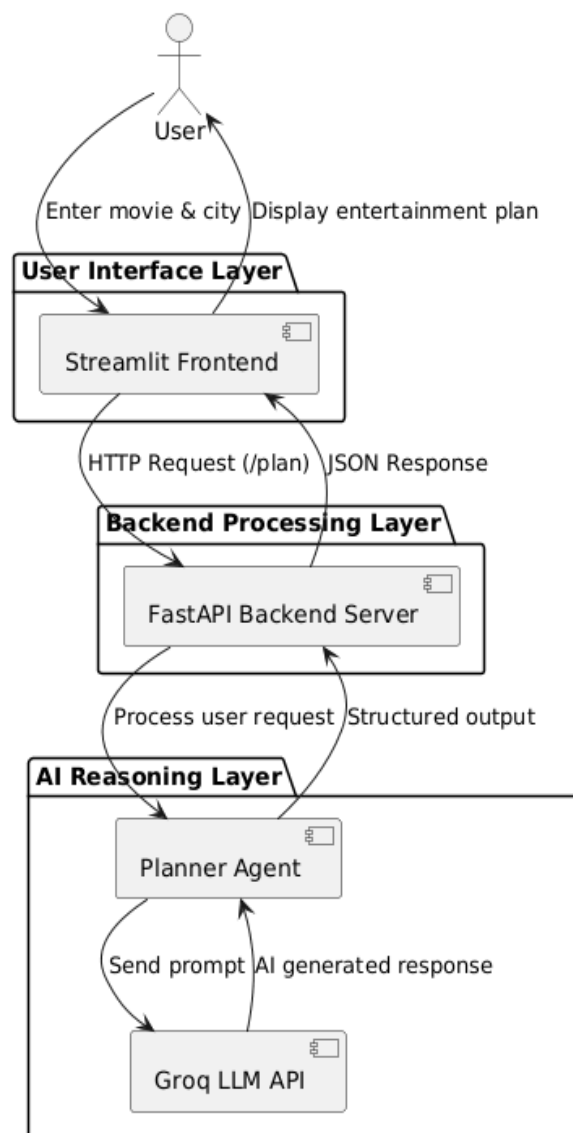
2. System Design

The system is designed using a modular architecture where each component performs a specific role. The architecture separates user interaction, backend processing, and AI reasoning into independent modules.

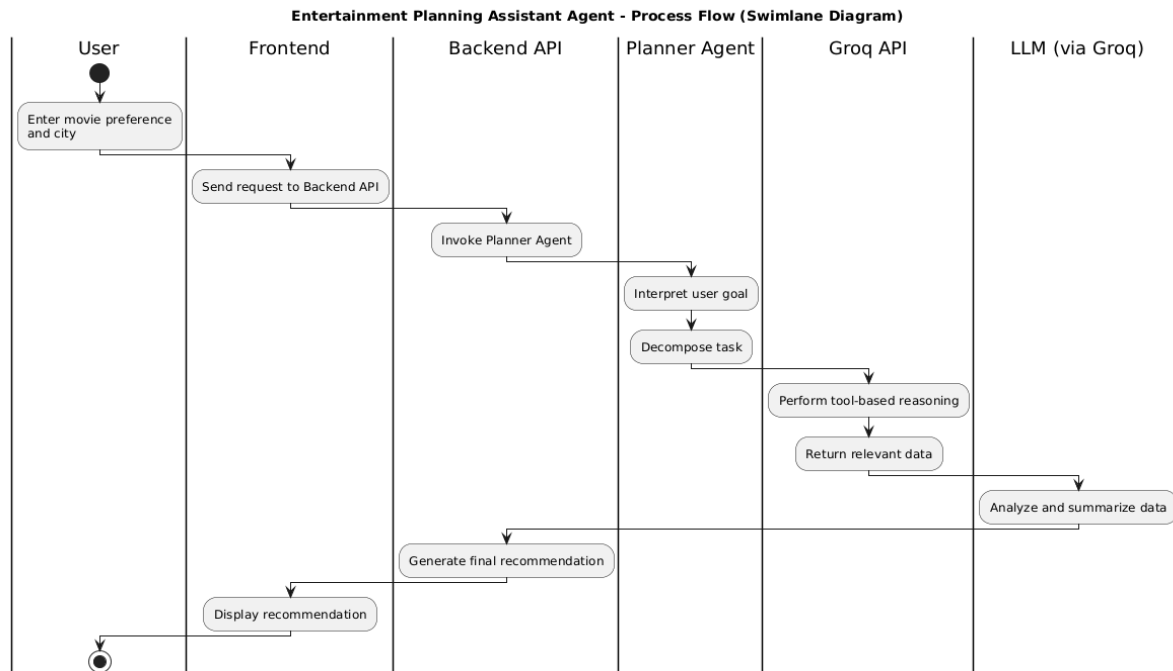
This modular design ensures that each component can be modified or extended without affecting other parts of the system.

2.1 Application Design

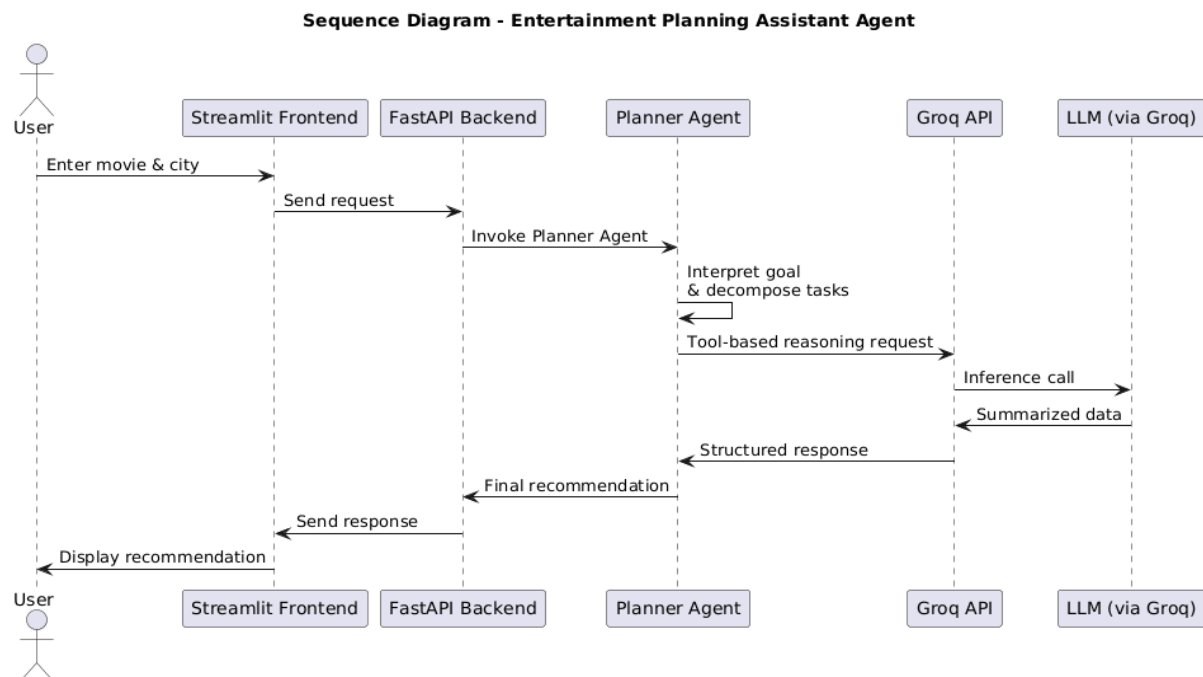
Entertainment Planning Assistant Agent - Application Design



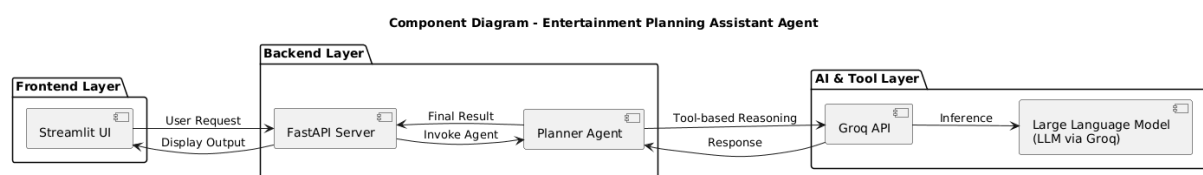
2.2 Process Flow



2.3 Information Flow



2.4 Components Design



2.5 Key Design Considerations

Several design considerations were taken into account while developing the system.

Modular Architecture

The system is designed with separate modules for frontend, backend, and AI logic to improve maintainability.

Scalability

The use of API-based architecture allows the system to be easily extended with additional features or services.

Performance

Using Groq LLM ensures fast response times compared to local language model execution.

Error Handling

The backend implements error handling mechanisms to manage API failures and invalid user inputs.

Security

API keys are stored using environment variables to prevent exposure of sensitive information.

2.6 API Catalogue

The system exposes REST API endpoints for communication between the frontend and backend.

Endpoint: GET /plan

Description:

This endpoint generates an entertainment recommendation based on the movie or event name and city provided by the user.

Request Parameters:

movie – Name of the movie or event

city – City where the user wants entertainment options

3. References

FastAPI Documentation

<https://fastapi.tiangolo.com>

Streamlit Documentation

<https://docs.streamlit.io>

Groq API Documentation

<https://console.groq.com/docs>

Python Official Documentation

<https://docs.python.org>